

SISTEMA DE ADQUISICIÓN DE CORRIENTE MEDIANTE RESISTENCIA SHUNT PARA ANÁLISIS DE UN MOTOR DC

Ingeniería Mecatrónica

Luis Fernando Lopez Calle¹
lopez.12636909@gmail.com
Yhoel Rojas Condori²
rojas.12542072@gmail.com
Ing. Vidher W. Bustillos D.³

Resumen

En el presente trabajo se desarrolló un sistema de adquisición de datos orientado a la medición de corriente eléctrica utilizando una resistencia shunt de 0.1 ohmios, un motor de corriente continua (DC) y un Arduino Mega. El sistema permite medir la caída de voltaje en la resistencia shunt generada por el paso de corriente hacia el motor, la cual es leída mediante el convertidor analógico–digital (ADC) del Arduino.

Los datos obtenidos son enviados a una computadora mediante comunicación serial, donde son procesados y almacenados para su posterior análisis. A partir de la ley de Ohm, se realizó la conversión de voltaje a corriente, permitiendo analizar el comportamiento del motor en función del consumo eléctrico.

Finalmente, se generaron representaciones gráficas que muestran la variación de la corriente en función del tiempo, evidenciando el comportamiento dinámico del motor DC. Los resultados demuestran la viabilidad de implementar sistemas de medición de corriente de bajo costo utilizando Arduino y resistencias shunt.

Palabras claves: Resistencia shunt, corriente eléctrica, Arduino Mega, motor DC, adquisición de datos, ley de Ohm.

¹ Autor de contacto: Estudiante de Ingeniería Mecatrónica, Universidad Mayor de San Andrés (UMSA), La Paz, Bolivia. Cursa la asignatura MEC-237: Laboratorio de Métodos Numéricos III. Correo electrónico: lopez.12636909@gmail.com

² Autor: Estudiante de Ingeniería Mecatrónica, Universidad Mayor de San Andrés (UMSA), La Paz, Bolivia. Cursa la asignatura MEC-237: Laboratorio de Métodos Numéricos III. Correo electrónico: rojas.12542072@gmail.com

³ Director: Se debe ubicar perfil académico del autor: los estudios realizados, la universidad, institución o grupo de investigación en el que se hallan inscritos, ya sea de manera académica o laboral.

Current Data Acquisition System Using a Shunt Resistor for DC Motor Analysis

Abstract

This work presents the development of a data acquisition system for measuring electric current using a 0.1 ohm shunt resistor, a DC motor, and an Arduino Mega. The system measures the voltage drop across the shunt resistor caused by the current flowing through the motor, which is read using the Arduino's analog-to-digital converter (ADC).

The acquired data are transmitted to a computer via serial communication, where they are processed and stored for further analysis. Using Ohm's law, the voltage measurements are converted into current values, allowing the analysis of motor behavior based on its electrical consumption.

Finally, graphical representations were generated to show current variation over time, demonstrating the dynamic behavior of the DC motor. The results confirm that low-cost current measurement systems can be implemented using Arduino and shunt resistors.

Keywords: shunt resistor, electric current, Arduino Mega, DC motor, data acquisition, Ohm's law.

Introducción o Antecedentes

En la ingeniería electrónica y mecatrónica, la medición de corriente eléctrica es fundamental para analizar el comportamiento de dispositivos eléctricos como motores, sensores y sistemas de potencia. Una de las técnicas más utilizadas para la medición de corriente es el uso de resistencias shunt, las cuales permiten convertir la corriente en una caída de voltaje proporcional.

El principio de funcionamiento se basa en la ley de Ohm, donde la corriente que atraviesa una resistencia puede determinarse a partir del voltaje medido en sus terminales. Esta técnica es ampliamente utilizada debido a su simplicidad, bajo costo y facilidad de implementación en sistemas de adquisición de datos.

Por otro lado, plataformas como Arduino permiten digitalizar señales analógicas mediante su convertidor ADC, facilitando la captura y procesamiento de datos en tiempo real. En el caso del Arduino Mega, este cuenta con una resolución de 10 bits, lo que permite obtener mediciones suficientemente precisas para aplicaciones de laboratorio.

En este contexto, el presente trabajo tiene como objetivo implementar un sistema de adquisición de datos para medir la corriente consumida por un motor DC utilizando una resistencia shunt y un Arduino Mega, analizando su comportamiento en función del tiempo.

Desarrollo de la Investigación

Solución propuesta

Se diseñó un sistema de medición de corriente utilizando una resistencia shunt de 0.1Ω conectada en serie con un motor DC alimentado por una fuente de 12 V.

El Arduino Mega mide el voltaje en los terminales de la resistencia shunt mediante una entrada analógica. Posteriormente, los datos son enviados a una computadora mediante comunicación serial para su procesamiento.

Aplicando la ley de Ohm, se obtiene la corriente:

$$I = \frac{V_{shunt}}{0.1}$$

Esto permite conocer en tiempo real el consumo de corriente del motor.

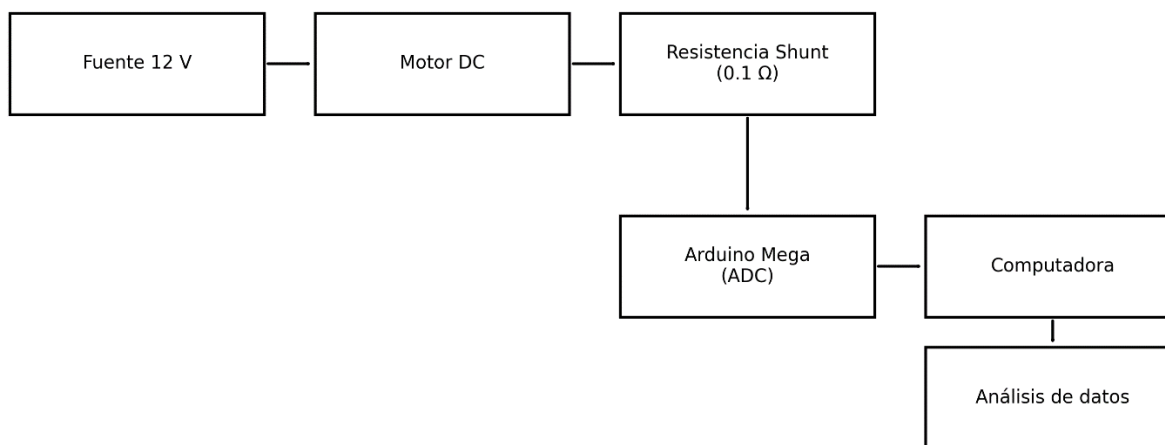
Requerimientos materiales y equipos

Para la implementación del sistema de adquisición de corriente se utilizaron los siguientes materiales y herramientas:

- Arduino Mega
- Motor de corriente continua (DC)
- Resistencia shunt de 0.1Ω
- Fuente de alimentación de 12 V
- Protoboard
- Cables de conexión
- Computadora personal
- Arduino IDE
- Software para visualización y procesamiento de datos

Diagrama en bloques

El diagrama en bloques del sistema de adquisición de corriente muestra el flujo de energía y de información desde la alimentación del motor hasta el procesamiento final de los datos. La fuente de 12 V suministra energía al motor DC, mientras que la resistencia shunt, conectada en serie, permite generar una caída de voltaje proporcional a la corriente del circuito. Dicha señal es adquirida por el Arduino Mega, procesada mediante su convertidor analógico–digital y enviada a una computadora por comunicación serial, donde los datos son registrados y analizados.



Codificación de los programas

Para el desarrollo del sistema de adquisición de corriente se utilizaron dos programas: uno implementado en **Arduino IDE** para la lectura de la señal analógica proveniente de la resistencia shunt, y otro desarrollado en **Python en Visual Studio Code** para la recepción, almacenamiento y representación gráfica de los datos adquiridos.

Fragmentos principales del código

a) Programa en Arduino IDE

El código cargado en el Arduino Mega tuvo como función principal realizar la lectura del voltaje presente en la resistencia shunt conectada al pin analógico **A0**. Para ello, se utilizó el convertidor analógico–digital de 10 bits del microcontrolador, considerando un voltaje de referencia de **5 V** y una resolución de **1023 niveles**.

A partir de la lectura analógica, el programa convierte el valor digital a voltaje mediante la expresión:

$$V_{shunt} = adc \cdot \frac{V_{ref}}{1023}$$

Posteriormente, aplicando la ley de Ohm, se calcula la corriente consumida por el motor:

$$I = \frac{V_{shunt}}{R_{shunt}}$$

donde $R_{shunt} = 0.1 \Omega$

Fragmento principal del código en Arduino

```
C++

#define VREF_MAX 5.0

#define RESOLUTION 1023.0

int sensor_shunt = A0;

float voltajeShunt = 0;

float corrienteMotor = 0;

float Rshunt = 0.1;

void setup() {

    pinMode(sensor_shunt, INPUT);

    Serial.begin(9600);

    Serial.println("t_ms,adc,Vshunt_V,I_A");

}

void loop() {

    int adc = analogRead(sensor_shunt);

    voltajeShunt = adc * (VREF_MAX / RESOLUTION);

    corrienteMotor = voltajeShunt / Rshunt;

    Serial.print(millis());

    Serial.print(",");
```

```
Serial.print(adc);

Serial.print(",");

Serial.print(voltajeShunt,5);

Serial.print(",");

Serial.println(corrienteMotor,5);

delay(100);

}
```

b) Programa en Python (Visual Studio Code)

El segundo programa fue desarrollado en Python y ejecutado en Visual Studio Code. Su función consistió en establecer la comunicación serial con el Arduino Mega, leer los datos enviados en tiempo real, almacenarlos en un archivo CSV y generar la gráfica de **corriente en función del tiempo**.

El script abre el puerto serial configurado a **9600 baudios**, espera la inicialización de la conexión y luego registra un número determinado de muestras. Cada línea recibida contiene cuatro variables: tiempo en milisegundos, valor ADC, voltaje en la resistencia shunt y corriente calculada.

Los datos adquiridos son almacenados en el archivo **datos_shunt.csv**, lo que permite su posterior análisis. Finalmente, con ayuda de la librería **Matplotlib**, se genera una gráfica de la corriente medida respecto al tiempo.

Fragmento principal del código en Python

```
PHYTON

import serial

import time

import csv

import matplotlib.pyplot as plt

PORT = "COM3"

BAUD = 9600

N_MUESTRAS = 150

ARCHIVO = "datos_shunt.csv"

ser = serial.Serial(PORT, BAUD, timeout=1)
```

```
time.sleep(2)

tiempo = []

corriente = []

with open(ARCHIVO, "w", newline="", encoding="utf-8") as f:

    writer = csv.writer(f)

    writer.writerow(["t_ms", "adc", "Vshunt_V", "I_A"])

    while len(tiempo) < N_MUESTRAS:

        line = ser.readline().decode(errors="ignore").strip()

        if not line or line.startswith("t_ms"):

            continue

        datos = line.split(",")

        if len(datos) == 4:

            t = float(datos[0])

            adc = float(datos[1])

            v = float(datos[2])

            iA = float(datos[3])

            writer.writerow([t, adc, v, iA])

            tiempo.append(t/1000)

            corriente.append(iA)

ser.close()

plt.plot(tiempo, corriente)

plt.xlabel("Tiempo (s)")

plt.ylabel("Corriente (A)")

plt.title("Corriente vs Tiempo")

plt.grid(True)

plt.show()
```

Resultados Obtenidos

Durante el desarrollo del experimento se registraron los datos de corriente consumida por un motor DC mediante una resistencia shunt de $0.1\ \Omega$ y un Arduino Mega. Las mediciones obtenidas fueron enviadas a una computadora a través de comunicación serial, almacenadas en un archivo CSV y posteriormente representadas gráficamente para analizar el comportamiento temporal de la corriente.

El archivo generado contiene cuatro variables principales: tiempo en milisegundos, lectura ADC, voltaje en la resistencia shunt y corriente calculada en amperios. A partir de estos datos fue posible observar el comportamiento dinámico del motor durante su arranque y funcionamiento.

Comportamiento de la corriente medida

La gráfica de corriente en función del tiempo mostró que, al momento del arranque, el motor presentó un **pico inicial elevado de corriente**, alcanzando aproximadamente **6 A**. Este comportamiento es característico de los motores de corriente continua, ya que en el instante de encendido demandan una corriente mayor para vencer la inercia mecánica y comenzar el movimiento.

Posteriormente, una vez superada la etapa de arranque, la corriente disminuyó y tendió a estabilizarse en un rango aproximado entre **3 A y 4 A**. Sin embargo, durante esta etapa no se observó un valor completamente constante, sino fluctuaciones considerables alrededor de ese rango.

Estas variaciones pueden atribuirse a diferentes factores, entre ellos:

- cambios en la carga mecánica del motor,
- vibraciones propias del sistema,
- pequeñas perturbaciones eléctricas,
- y variaciones en la medición debidas al ruido de la señal adquirida.

Registro de datos experimentales

En la **Tabla 1** se presenta un fragmento representativo de los datos registrados durante el experimento. Estos valores fueron obtenidos directamente del archivo CSV generado por el sistema de adquisición de datos de corriente.

Tabla 1. Fragmento de los datos registrados durante la medición de corriente mediante resistencia shunt.

t_ms,adc,Vshunt_V,I_A
0.0,0.0,0.0,0.0
101.0,0.0,0.0,0.0

202.0,0.0,0.0,0.0
305.0,0.0,0.0,0.0
406.0,0.0,0.0,0.0
507.0,0.0,0.0,0.0
609.0,0.0,0.0,0.0
711.0,0.0,0.0,0.0
813.0,0.0,0.0,0.0
914.0,0.0,0.0,0.0
1015.0,0.0,0.0,0.0
1118.0,0.0,0.0,0.0
1219.0,0.0,0.0,0.0
1320.0,126.0,0.61584,6.15836
1423.0,85.0,0.41544,4.15445
1524.0,61.0,0.29814,2.98143
1627.0,63.0,0.30792,3.07918
1728.0,60.0,0.29326,2.93255
1830.0,57.0,0.27859,2.78592
1932.0,56.0,0.2737,2.73705
2034.0,60.0,0.29326,2.93255
2136.0,57.0,0.27859,2.78592
2238.0,52.0,0.25415,2.54154
2339.0,49.0,0.23949,2.39492
2442.0,53.0,0.25904,2.59042
2543.0,40.0,0.1955,1.95503
2646.0,50.0,0.24438,2.44379
2747.0,53.0,0.25904,2.59042
2848.0,56.0,0.2737,2.73705
2951.0,52.0,0.25415,2.54154
3052.0,53.0,0.25904,2.59042
3154.0,50.0,0.24438,2.44379
3256.0,56.0,0.2737,2.73705
3358.0,54.0,0.26393,2.6393
3460.0,50.0,0.24438,2.44379
3562.0,55.0,0.26882,2.68817
3663.0,56.0,0.2737,2.73705
3766.0,51.0,0.24927,2.49267
3867.0,64.0,0.31281,3.12805
3970.0,45.0,0.21994,2.19941
4071.0,55.0,0.26882,2.68817
4173.0,54.0,0.26393,2.6393
4275.0,53.0,0.25904,2.59042
4376.0,49.0,0.23949,2.39492
4478.0,45.0,0.21994,2.19941
4580.0,49.0,0.23949,2.39492

4682.0,50.0,0.24438,2.44379
4784.0,46.0,0.22483,2.24829
4886.0,48.0,0.2346,2.34604
4987.0,54.0,0.26393,2.6393
5090.0,49.0,0.23949,2.39492
5191.0,44.0,0.21505,2.15054
5294.0,44.0,0.21505,2.15054
5395.0,48.0,0.2346,2.34604
5497.0,41.0,0.20039,2.00391
5599.0,46.0,0.22483,2.24829
5701.0,45.0,0.21994,2.19941
5803.0,51.0,0.24927,2.49267
5904.0,53.0,0.25904,2.59042
6006.0,46.0,0.22483,2.24829
6108.0,44.0,0.21505,2.15054
6210.0,46.0,0.22483,2.24829
6311.0,50.0,0.24438,2.44379
6414.0,43.0,0.21017,2.10166
6515.0,43.0,0.21017,2.10166
6618.0,48.0,0.2346,2.34604
6719.0,38.0,0.18573,1.85728
6821.0,48.0,0.2346,2.34604
6923.0,48.0,0.2346,2.34604
7025.0,43.0,0.21017,2.10166
7127.0,52.0,0.25415,2.54154
7229.0,43.0,0.21017,2.10166
7330.0,48.0,0.2346,2.34604
7432.0,63.0,0.30792,3.07918
7534.0,41.0,0.20039,2.00391
7635.0,44.0,0.21505,2.15054
7738.0,51.0,0.24927,2.49267
7839.0,49.0,0.23949,2.39492
7942.0,49.0,0.23949,2.39492
8043.0,42.0,0.20528,2.05279
8145.0,45.0,0.21994,2.19941
8247.0,51.0,0.24927,2.49267
8349.0,47.0,0.22972,2.29717
8451.0,60.0,0.29326,2.93255
8553.0,44.0,0.21505,2.15054
8654.0,42.0,0.20528,2.05279
8756.0,50.0,0.24438,2.44379
8858.0,47.0,0.22972,2.29717
8960.0,49.0,0.23949,2.39492
9062.0,57.0,0.27859,2.78592

9163.0,44.0,0.21505,2.15054
9266.0,42.0,0.20528,2.05279
9367.0,45.0,0.21994,2.19941
9469.0,51.0,0.24927,2.49267
9571.0,50.0,0.24438,2.44379
9673.0,44.0,0.21505,2.15054
9775.0,56.0,0.2737,2.73705
9877.0,45.0,0.21994,2.19941
9978.0,36.0,0.17595,1.75953
10081.0,49.0,0.23949,2.39492
10182.0,54.0,0.26393,2.6393
10285.0,41.0,0.20039,2.00391
10386.0,45.0,0.21994,2.19941
10488.0,45.0,0.21994,2.19941
10590.0,42.0,0.20528,2.05279
10692.0,44.0,0.21505,2.15054
10793.0,45.0,0.21994,2.19941
10896.0,41.0,0.20039,2.00391
10997.0,43.0,0.21017,2.10166
11100.0,42.0,0.20528,2.05279
11201.0,44.0,0.21505,2.15054
11303.0,44.0,0.21505,2.15054
11405.0,44.0,0.21505,2.15054
11507.0,40.0,0.1955,1.95503
11609.0,51.0,0.24927,2.49267
11711.0,51.0,0.24927,2.49267
11812.0,46.0,0.22483,2.24829
11915.0,49.0,0.23949,2.39492
12016.0,44.0,0.21505,2.15054
12119.0,43.0,0.21017,2.10166
12220.0,48.0,0.2346,2.34604
12322.0,50.0,0.24438,2.44379
12424.0,42.0,0.20528,2.05279
12526.0,37.0,0.18084,1.80841
12627.0,50.0,0.24438,2.44379
12730.0,32.0,0.1564,1.56403
12831.0,42.0,0.20528,2.05279
12934.0,61.0,0.29814,2.98143
13035.0,46.0,0.22483,2.24829
13137.0,48.0,0.2346,2.34604
13239.0,57.0,0.27859,2.78592
13341.0,40.0,0.1955,1.95503
13443.0,48.0,0.2346,2.34604
13545.0,37.0,0.18084,1.80841

13646.0,41.0,0.20039,2.00391
13749.0,43.0,0.21017,2.10166
13850.0,40.0,0.1955,1.95503
13953.0,49.0,0.23949,2.39492
14054.0,48.0,0.2346,2.34604
14156.0,44.0,0.21505,2.15054
14258.0,55.0,0.26882,2.68817
14360.0,40.0,0.1955,1.95503
14461.0,42.0,0.20528,2.05279
14564.0,41.0,0.20039,2.00391
14665.0,37.0,0.18084,1.80841
14768.0,39.0,0.19062,1.90616
14869.0,50.0,0.24438,2.44379
14971.0,44.0,0.21505,2.15054
15073.0,40.0,0.1955,1.95503
15175.0,38.0,0.18573,1.85728

***Nota:** Los datos presentados corresponden a las mediciones obtenidas durante el experimento de adquisición de corriente utilizando una resistencia shunt conectada en serie con un motor DC y un Arduino Mega. Las mediciones fueron registradas mediante un programa desarrollado en Python y almacenadas en un archivo CSV para su posterior análisis.*

Representación gráfica de los datos

Interpretación de la gráfica

La respuesta obtenida evidencia dos zonas claramente diferenciadas:

1. **Zona de arranque:** caracterizada por un consumo elevado de corriente, con un pico cercano a 6 A.
2. **Zona de régimen de funcionamiento:** caracterizada por una corriente menor, alrededor de 3 A a 4 A, aunque con oscilaciones notables.

Este comportamiento confirma que el sistema implementado fue capaz de capturar adecuadamente tanto el transitorio de arranque como el régimen variable de operación del motor DC.

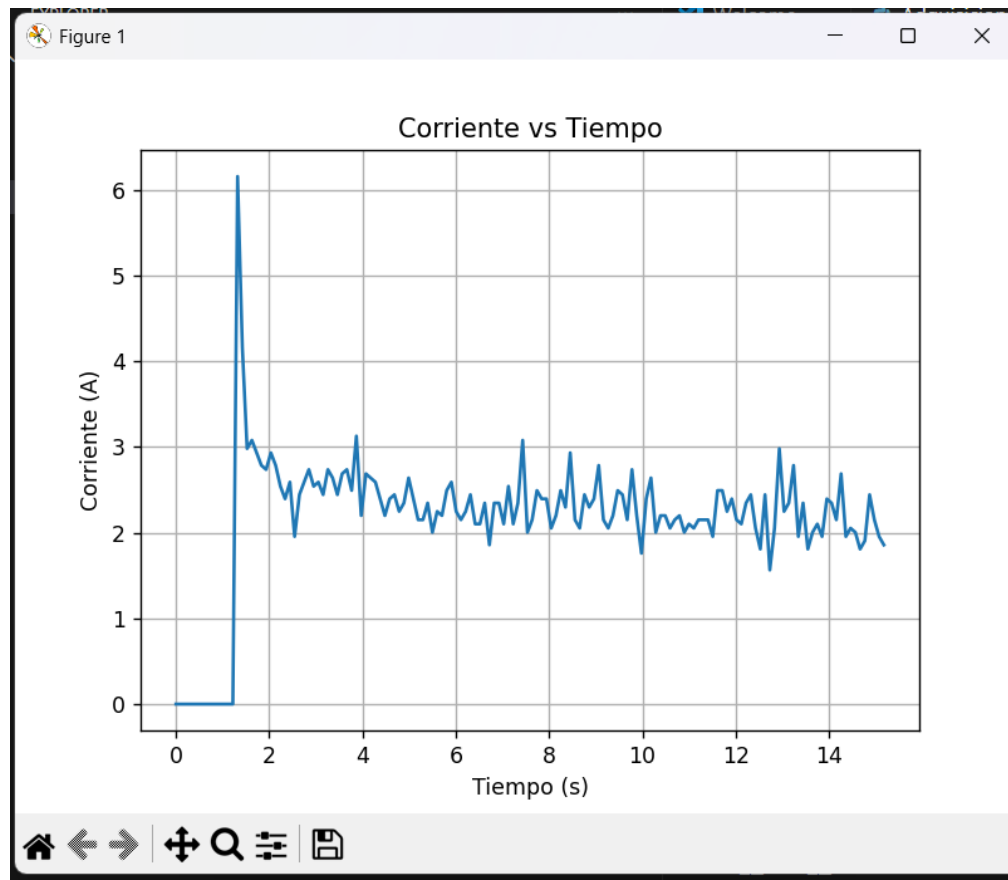


Figura 2. Variación de la corriente del motor DC en función de las muestras registradas durante el experimento.

Fuente: Elaboración propia a partir de los datos obtenidos durante el experimento de medición de corriente utilizando Arduino Mega y resistencia shunt.

Discusión y conclusiones

Discusión de resultados

Los resultados obtenidos permiten analizar el comportamiento del sistema de medición de corriente basado en una resistencia shunt, así como su relación con los métodos de medición invasivos y no invasivos.

El uso de la resistencia shunt constituye un **método invasivo de medición**, ya que este elemento se conecta en serie con la carga (motor DC), alterando ligeramente el circuito original al introducir una caída de voltaje proporcional a la corriente. A pesar de ello, este método presenta la ventaja de ser simple, económico y de fácil implementación.

Durante el experimento se observó un pico inicial de corriente cercano a 6 A en el arranque del motor, seguido de una estabilización en el rango de 3 A a 4 A. Estos valores se reflejan directamente en la caída de voltaje en la resistencia shunt, lo que confirma la relación lineal establecida por la ley de Ohm entre corriente y voltaje.

Las fluctuaciones observadas en la señal de corriente pueden atribuirse tanto al comportamiento dinámico del motor como a factores asociados al sistema de medición, tales como ruido eléctrico y la resolución del convertidor analógico–digital del Arduino.

En contraste, los métodos no invasivos, como el uso de sensores basados en el efecto Hall, permiten medir la corriente sin necesidad de interrumpir el circuito ni introducir una caída de voltaje. Sin embargo, estos sistemas suelen ser más costosos y requieren una calibración más compleja en comparación con la medición mediante resistencia shunt.

En este sentido, el experimento permitió evidenciar las ventajas y limitaciones del método invasivo, destacando su utilidad en aplicaciones de laboratorio donde la simplicidad y el bajo costo son factores importantes.

Conclusiones

El desarrollo del presente laboratorio permitió implementar un sistema de adquisición de corriente utilizando una resistencia shunt y un Arduino Mega, cumpliendo el objetivo de medir y analizar el consumo de corriente de un motor DC.

Se comprobó que la resistencia shunt permite obtener una medición indirecta de la corriente mediante la conversión de esta en una señal de voltaje proporcional, validando la aplicación práctica de la ley de Ohm en sistemas de adquisición de datos.

Asimismo, se evidenció que el método utilizado corresponde a una técnica invasiva de medición, la cual, si bien introduce una pequeña perturbación en el circuito, resulta altamente eficiente, precisa y de bajo costo para aplicaciones experimentales.

El análisis comparativo con métodos no invasivos, como los sensores de efecto Hall, permitió comprender que estos últimos ofrecen la ventaja de no alterar el circuito, aunque implican mayor complejidad y costo.

Finalmente, se concluye que el sistema implementado es una herramienta adecuada para la medición de corriente en entornos educativos, permitiendo no solo adquirir datos en tiempo real, sino también comprender los principios fundamentales de instrumentación y adquisición de señales eléctricas.

Referencias bibliográficas

Referencias

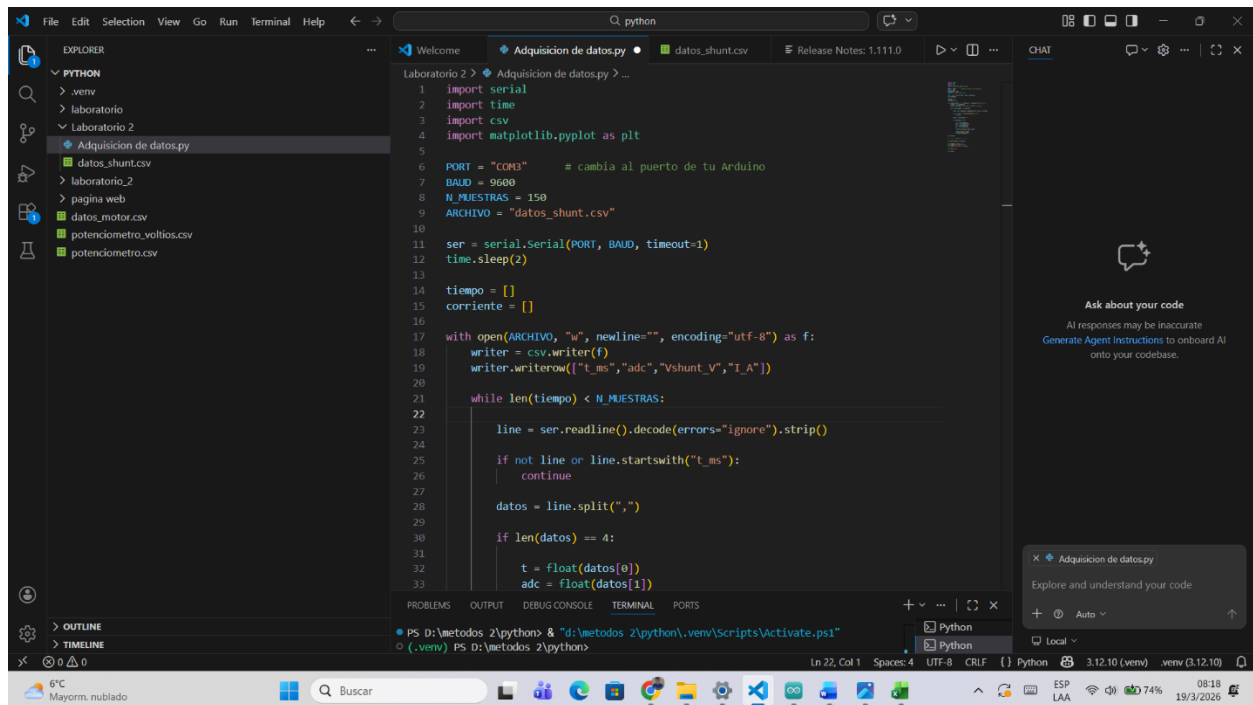
Arduino. (2023). *Arduino Mega 2560 Rev3 – Technical specifications*. Arduino Documentation. <https://www.arduino.cc/en/Main/ArduinoBoardMega2560>

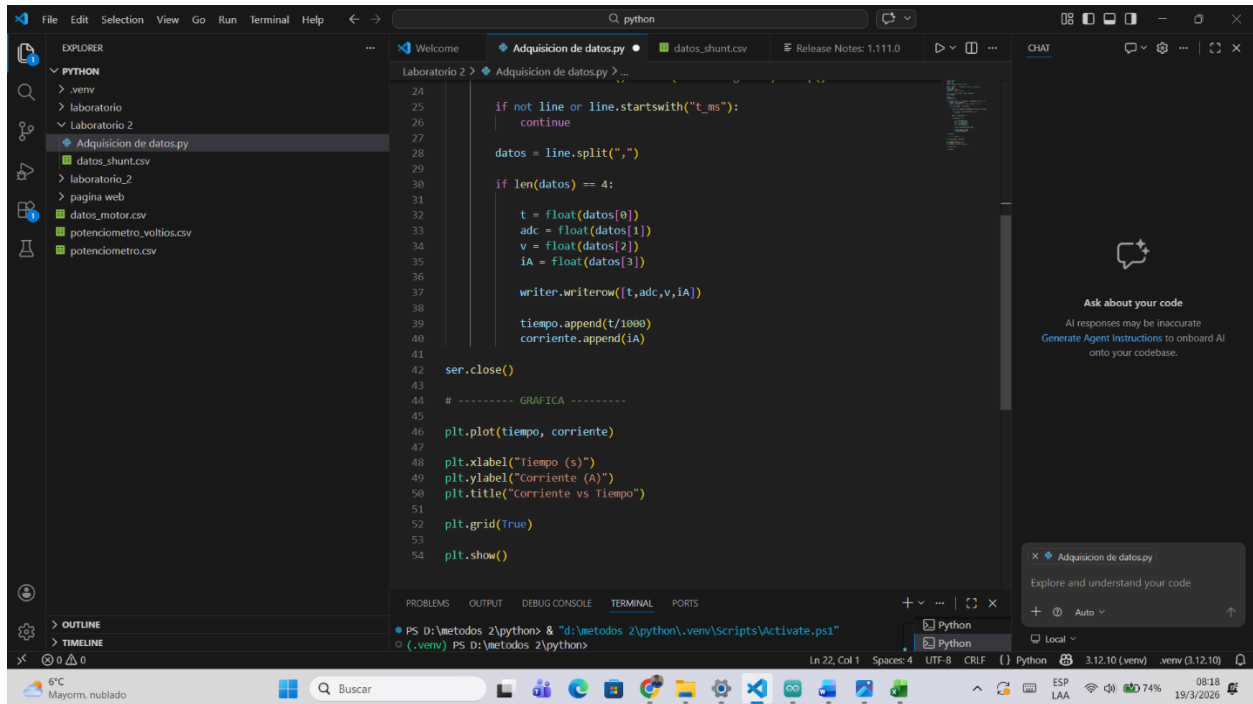
Alexander, C. K., & Sadiku, M. N. O. (2013). *Fundamentos de circuitos eléctricos* (5.a ed.). McGraw-Hill.

Python Software Foundation. (2023). *Python language reference*. <https://docs.python.org/3/>

Matplotlib Development Team. (2023). *Matplotlib documentation*. <https://matplotlib.org/stable/index.html>

ANEXOS



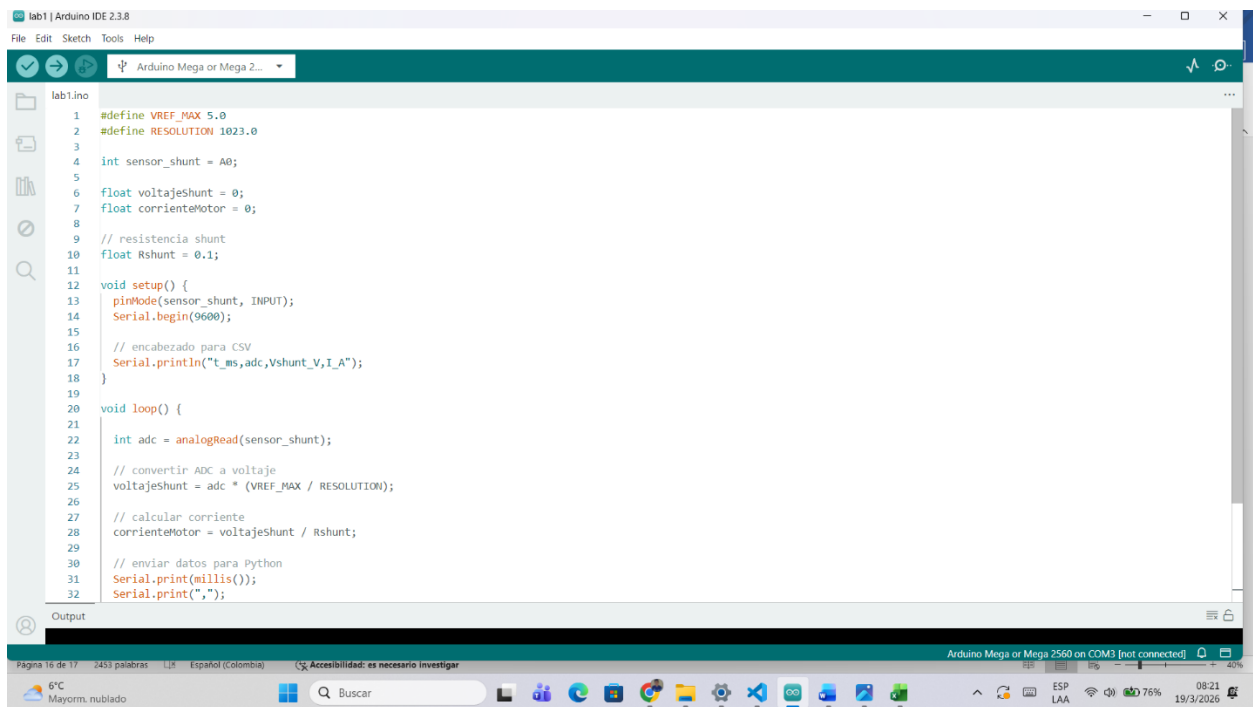


The screenshot shows a Visual Studio Code editor window with a Python file named 'Adquisición de datos.py'. The code is a script for data acquisition, likely from an Arduino. It includes comments in Spanish and Python code for reading data from a serial port, processing it, and plotting it. The code is as follows:

```
24
25
26 if not line or line.startswith("\n"):
27     continue
28
29 datos = line.split(",")
30
31 if len(datos) == 4:
32     t = float(datos[0])
33     adc = float(datos[1])
34     v = float(datos[2])
35     IA = float(datos[3])
36
37     writer.writerow([t,adc,v,IA])
38
39 tiempo.append(t/1000)
40 corriente.append(IA)
41
42 ser.close()
43
44 # ----- GRAFICA -----
45
46 plt.plot(tiempo, corriente)
47
48 plt.xlabel("Tiempo (s)")
49 plt.ylabel("Corriente (A)")
50 plt.title("Corriente vs Tiempo")
51
52 plt.grid(True)
53
54 plt.show()
```

On the right side of the editor, there is a chat window titled 'Ask about your code'. It contains the text: 'AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase.'

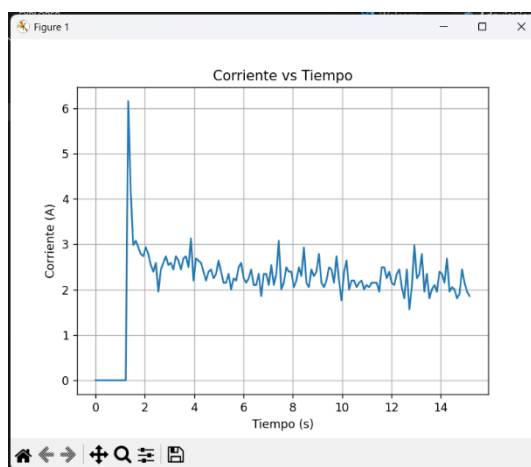
The bottom status bar shows the file is open in the 'Python' environment, with the path 'D:\metodos 2\python> & "d:\metodos 2\python\.venv\Scripts\Activate.ps1"'. The system tray at the bottom shows the date and time as 08:18 19/3/2026.



The screenshot shows the Arduino IDE interface with a C++ sketch for data acquisition. The sketch is named 'lab1.ino' and is for an Arduino Mega or Mega 2560. The code is as follows:

```
1 #define VREF_MAX 5.0
2 #define RESOLUTION 1023.0
3
4 int sensor_shunt = A0;
5
6 float voltajeShunt = 0;
7 float corrienteMotor = 0;
8
9 // resistencia shunt
10 float Rshunt = 0.1;
11
12 void setup() {
13     pinMode(sensor_shunt, INPUT);
14     Serial.begin(9600);
15
16     // encabezado para CSV
17     Serial.println("t_ms,adc,Vshunt_V,I_A");
18 }
19
20 void loop() {
21
22     int adc = analogRead(sensor_shunt);
23
24     // convertir ADC a voltaje
25     voltajeShunt = adc * (VREF_MAX / RESOLUTION);
26
27     // calcular corriente
28     corrienteMotor = voltajeShunt / Rshunt;
29
30     // enviar datos para Python
31     Serial.print(millis());
32     Serial.print(",");
```

The bottom status bar shows the board is 'Arduino Mega or Mega 2560 on COM3 [not connected]'. The system tray at the bottom shows the date and time as 08:21 19/3/2026.



1	t_ms,adc,Vshunt_V,I_A
34	3256.0,56.0,0.2737,2.73705
35	3358.0,54.0,0.26393,2.6393
36	3460.0,50.0,0.24438,2.44379
37	3562.0,55.0,0.26882,2.68817
38	3663.0,56.0,0.2737,2.73705
39	3766.0,51.0,0.24927,2.49267
40	3867.0,64.0,0.31281,3.12805
41	3970.0,45.0,0.21994,2.19941
42	4071.0,55.0,0.26882,2.68817
43	4173.0,54.0,0.26393,2.6393
44	4275.0,53.0,0.25904,2.59042
45	4376.0,49.0,0.23949,2.39492
46	4478.0,45.0,0.21994,2.19941
47	4580.0,49.0,0.23949,2.39492
48	4682.0,50.0,0.24438,2.44379
49	4784.0,46.0,0.22483,2.24829
50	4886.0,48.0,0.2346,2.34604
51	4987.0,54.0,0.26393,2.6393
52	5090.0,49.0,0.23949,2.39492
53	5191.0,44.0,0.21505,2.15054
54	5294.0,44.0,0.21505,2.15054
55	5395.0,48.0,0.2346,2.34604
56	5497.0,41.0,0.20039,2.00391
57	5599.0,46.0,0.22483,2.24829
58	5701.0,45.0,0.21994,2.19941
59	5803.0,51.0,0.24927,2.49267
60	5904.0,53.0,0.25904,2.59042
61	6006.0,46.0,0.22483,2.24829
62	6108.0,44.0,0.21505,2.15054
63	6210.0,46.0,0.22483,2.24829
64	6311.0,50.0,0.24438,2.44379
65	6414.0,43.0,0.21017,2.10166

97	9673.0,44.0,0.21505,2.15054
98	9775.0,56.0,0.2737,2.73705
99	9877.0,45.0,0.21994,2.19941
100	9978.0,36.0,0.17595,1.75953
101	10081.0,49.0,0.23949,2.39492
102	10182.0,54.0,0.26393,2.6393
103	10285.0,41.0,0.20039,2.00391
104	10386.0,45.0,0.21994,2.19941
105	10488.0,45.0,0.21994,2.19941
106	10590.0,42.0,0.20528,2.05279
107	10692.0,44.0,0.21505,2.15054
108	10793.0,45.0,0.21994,2.19941
109	10896.0,41.0,0.20039,2.00391
110	10997.0,43.0,0.21017,2.10166
111	11100.0,42.0,0.20528,2.05279
112	11201.0,44.0,0.21505,2.15054
113	11303.0,44.0,0.21505,2.15054
114	11405.0,44.0,0.21505,2.15054
115	11507.0,40.0,0.1955,1.95503
116	11609.0,51.0,0.24927,2.49267
117	11711.0,51.0,0.24927,2.49267
118	11812.0,46.0,0.22483,2.24829
119	11915.0,49.0,0.23949,2.39492
120	12016.0,44.0,0.21505,2.15054
121	12119.0,43.0,0.21017,2.10166
122	12220.0,48.0,0.2346,2.34604
123	12322.0,50.0,0.24438,2.44379
124	12424.0,42.0,0.20528,2.05279
125	12526.0,37.0,0.18084,1.80841
126	12627.0,50.0,0.24438,2.44379
127	12730.0,32.0,0.1564,1.56403
128	12831.0,42.0,0.20528,2.05279